# Report for the Statistical Machine Learning (CES575)

Huajian Liu
*Arizona State University*
Tempe, US
*huajianl@asu.edu*

**Abstract—This report is intended to summarize the three projects from the Statistical Machine Learning course.**

## I. Introduction

The projects of this course covered implementations from three perspectives: supervised learning, unsupervised learning, and neural network learning. The specific implementations included density estimation and classification, implementation of the k-means algorithm, and classification using neural networks and deep learning. The following provides a detailed overview of each implementation.

## II. Project 1: Implementation of Density Estimation and Classification

### A. Project Description

In this project, we conducted parameter estimation and classification tasks on a given dataset containing samples of the digits "7" and "8". We split the dataset into training and testing sets, with both sets containing image samples of the digits "7" and "8". We assumed that the two features are independent of each other, and each sample drawn follows a two-dimensional normal distribution. Therefore, during data preprocessing, we extracted two features from each image: the mean value and the standard deviation.

After preprocessing the data, we implemented Naive Bayes classifier model and logistic regression model. We then trained both models separately and evaluated the accuracy of the two trained models using the testing data.

### B. Implementation of the Naive Bayes classifier Model

We create a class to implement the Naive Bayes classifier Model.The class defines several methods.

- `train:`This method trains the Naive Bayes classifier model using the given training dataset and corresponding labels. It calculates the mean, covariance, inverse covariance, determinant, and prior probabilities for both classes.
- `prob`: This method computes the probability of an input digit belonging to a specific class (either "7" or "8"). It utilizes the mean, covariance, and prior probabilities calculated during training to estimate the likelihood of the input digit belonging to each class.
- `predict`: This method predicts the class labels for a set of input digits. It iterates through each input digit and determines its predicted class based on the probabilities computed using the `prob` method.The implementation employs NumPy for efficient array operations, such as calculating mean, covariance, and matrix multiplication. Overall, this Naive Bayes classifier model provides a straightforward yet effective approach for classifying digits based on their features.

### C. Implementation of the Logistic Regression Model

We create a class to implement the Logistic Regression Model.The class defines several methods.

- `sigma`: This method computes the sigmoid function, which maps any real-valued number to the range [0, 1], producing a probability.
- `calinp`: This method calculates the linear combination of input features and model weights, along with a bias term.
- `loss`: This method computes the loss function, which measures the difference between the predicted and true class labels, and calculates the gradient of the loss function with respect to the model parameters.
- `train`: This method trains the logistic regression model by iteratively updating the model parameters (weights and bias) using gradient descent to minimize the loss function.
- `predict`: This method predicts the class labels for input samples by applying the trained logistic regression model and using a threshold (0.5) to convert the sigmoid output to binary class labels.

### D. Results Analysis

Following the implementation and testing of the Naive Bayes classifier and logistic regression models, we analyzed the results to evaluate their performance in classifying the digits "7" and "8" based on the extracted features.

TABLE I. Model Comparison Results

| **Model Name** | **Indicator Name** | | |
|---|---|---|---|
| | ***overall accuracy*** | ***accuracy of 7*** | ***accuracy of 8*** |
| *Naive Bayes* | 0.7 | 0.76 | 0.63 |
| *Logistic Regression* | 0.81 | 0.81 | 0.81 |

The results are rounded to two decimal places using rounding rules (rounding half up).

### E. What have we learned

From the results, it appears that the accuracy of the logistic regression model is better than that of the Naive Bayes model. However, both models do not exhibit very high

accuracy in identifying the data.Considering the characteristics of the data, the features of mean and variance are correlated. However, the Naive Bayes model assumes that the features are independent of each other. From this perspective, selecting these two features could affect the accuracy of the Bayesian model.And the data samples for 7 and 8 exhibit significant differences in terms of mean and variance features. Because the logistic regression model is essentially a linear model, it can better classify this dataset.

## III. Project 2: Implementation of K-means (Unsupervised learning )

### A. Project Description

The project implemented the K-means model and used it for unsupervised learning classification of the data samples.K-means clustering is a method of vector quantization, originally from signal processing, that aims to partition n observations into k clusters in which each observation belongs to the cluster with the nearest mean (cluster centers or cluster centroid), serving as a prototype of the cluster.

### B. Implementation of the K-Means

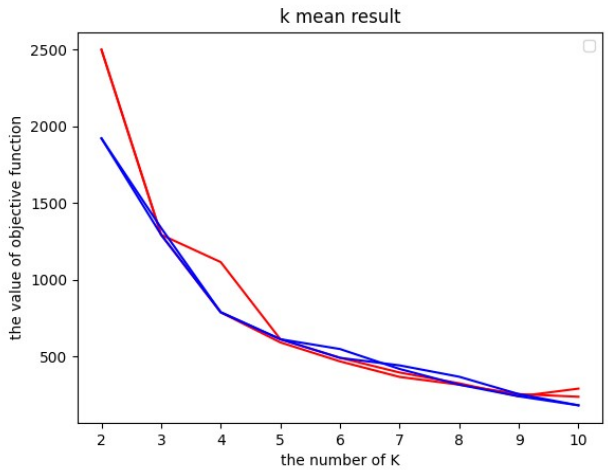
We create a class to implement the k-Means clustering algorithm, which supports two strategies for initializing cluster centroids. The class is initialized with parameters including the number of clusters and the dataset.

- `initiate_centroids_randomly:method implements the k-Means clustering algorithm, iteratively assigning data points to the nearest centroid and updating centroids until convergence is achieved. It calculates distances between data points and centroids, assigns each data point to the nearest centroid, computes new centroids based on the mean of data points in each cluster, and repeats this process until convergence.`
- `initiate_centroids_farthest`:This method initializes cluster centroids based on the farthest distance strategy, where centroids are selected such that they are as far apart from each other as possible.
- `culstering`: This method implements the k-Means clustering algorithm, iteratively assigning data points to the nearest centroid and updating centroids until convergence is achieved. It calculates distances between data points and centroids, assigns each data point to the nearest centroid, computes new centroids based on the mean of data points in each cluster, and repeats this process until convergence.

### C. Results Analysis

We use a objective function measures the sum of distances between each data point and its assigned centroid. This calculation is performed by iterating over each centroid and each data point, checking if the data point belongs to the cluster associated with the current centroid, and then adding the squared Euclidean distance between the data point and the centroid to the result. Finally, the function returns the total sum of distances as the objective function value. This objective function is crucial in evaluating the effectiveness of the clustering algorithm and used to optimize the choice of cluster centroids during the iterative process of k-Means clustering.

Image 1 prject2 results




The graph illustrates the relationship between the number of clusters (k) and the objective function, with four curves representing different initialization strategies. The red curve represents the results of selecting initial centroids randomly, while the blue curve represents the results of selecting initial centroids based on the farthest distance strategy. Notably, all curves exhibit their maximum derivatives at k=4, indicating the fastest rate of decrease in the objective function value. This suggests that beyond k=4, the rate of decrease in the objective function value gradually slows down. This observation provides valuable insights into the optimal choice of the number of clusters, highlighting k=4 as a point where the clustering model achieves significant improvement before experiencing diminishing returns with further increases in the number of clusters.

### D. What have we learned

K-means algorithm operates by iteratively seeking the minimum distance between data points and cluster centers to partition the dataset into a predetermined number of tightly clustered groups. Due to its algorithmic characteristics, it is more suitable for classifying data with distinct cluster structures, a known number of clusters, or relatively simple data structures. For instance, in real-life applications, the K-means algorithm can be employed to cluster users based on their behavior and interests, identifying groups of users with similar patterns. This allows for personalized recommendations, such as friend or ad recommendations, tailored to different user groups. Additionally, in cloud computing, K-means clustering can be utilized to categorize operational issues, enabling quick identification of the root causes of problems by operations teams.

## IV. Project 3: Implementation of CNN model

### A. Project Description

The project implements a Convolutional Neural Network (CNN) model, which can recognize target digit images through a trained model. Convolutional Neural Networks (CNNs) are a type of deep learning model primarily used for processing data with grid structures, such as image data.

Structurally, it can be divided into convolutional layers, pooling layers, activation functions, and fully connected layers, loss functions. The implementation of each layer has a different degree of impact on the model's results. The project mainly experiments with kernel size and the number of features in convolutional layers to observe the effects of these two attributes on the results.

### *B. Implementation of the CNN model*

The code snippet begins by defining several variables including the batch size for gradient descent iterations, the number of classes, and the number of training epochs. It then loads the MNIST dataset and reshapes the data according to the specified image dimensions and format.The pixel values of the images are normalized to the range [0,1].

Following the data preprocessing steps, we defines a CNN model using the Sequential API provided by the Keras library. The model consists of multiple layers including convolutional layers, max-pooling layers, flattening layer, and fully connected layers. Each convolutional layer is followed by a ReLU activation function and a max-pooling layer to reduce the spatial dimensions of the feature maps. The output layer uses softmax activation to produce probability distributions over the classes.

After defining the model architecture, the code compiles the model using categorical cross-entropy as the loss function and the Adadelta optimizer with a specified learning rate. Then, it trains the model on the provided training data for a specified number of epochs, using a specified batch size for each iteration. During training, it also validates the model on a separate validation dataset.

After training, the code retrieves the training and validation loss, as well as the training and validation accuracy, for each epoch. These metrics wil be used to evaluate the performance of the model over the training period.

TABLE II. Experimental Parameter Settings

| **Experiment Number** | **Parameter Name** | | |
|---|---|---|---|
| | ***Kernel size*** | ***The number of features in the first convolutional layer.*** | ***The number of features in the second convolutional layer.*** |
| *first* | 3*3 | 6 | 16 |
| *second* | 5*5 | 6 | 16 |
| *third* | 5*5 | 10 | 24 |

### *C. Results Analysis*

The result segment visualizes the training and validation performance of the CNN model by plotting the loss and accuracy metrics over the epochs.

In the first subplot, it plots the training loss ('Loss') in red and the validation loss ('Validation-loss') in blue against the number of epochs. This allows us to observe how well the model is minimizing its loss function during training and how it performs on unseen validation data.

In the second subplot, it plots the training accuracy ('Accuracy') in red and the validation accuracy ('Validation-accuracy') in blue against the number of epochs. This enables us to track how well the model's predictions align with the actual labels both during training and on the validation set.

Image2 results of the first expriment

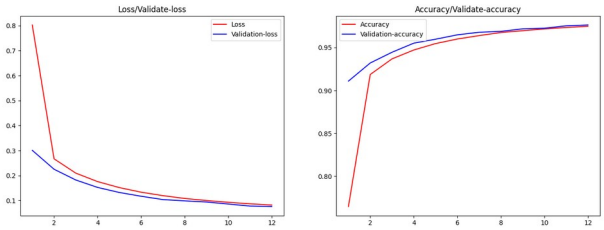



Image3 results of the second expriment

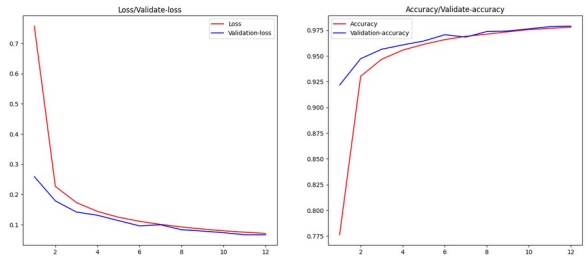
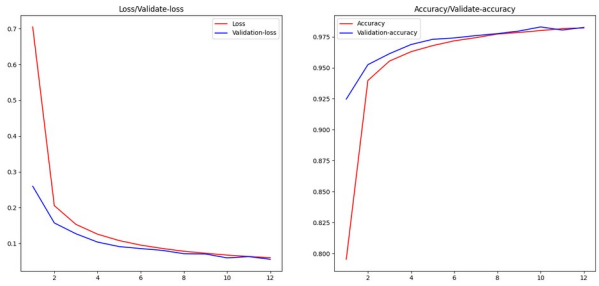



Image4 results of the third expriment




We can summarize the following points from the above results:

- CNNs are inherently well-suited for image classification tasks due to their ability to capture hierarchical features within the data. This enables them to effectively discern patterns and structures within handwritten digits, leading to accurate recognition results.
- Enlarging the size of the convolutional kernels in the CNN architecture expands the receptive field of the model, enabling it to capture more intricate details and spatial relationships within the input images. This augmentation of the receptive field empowers the model to extract more discriminative features, ultimately enhancing its accuracy in digit recognition.
- Augmenting the number of feature maps in the convolutional layers enriches the model's capacity to learn diverse representations of the input data. By increasing the complexity and diversity of features extracted from the images, the model becomes more adept at capturing the nuanced variations present in handwritten digits, consequently improving its accuracy in classification tasks.

### *D. What have we learned*

Convolutional Neural Networks (CNNs) find extensive applications across various domains due to their exceptional ability to process and extract features from structured data like images and videos. In computer vision, CNNs are widely employed for tasks such as image classification, object detection, facial recognition, and semantic segmentation. They are instrumental in medical imaging for diagnosing

diseases from scans like X-rays, MRIs, and CT scans. CNNs also play a crucial role in autonomous vehicles for tasks like lane detection, pedestrian detection, and traffic sign recognition. In the field of natural language processing, CNNs are utilized for text classification, sentiment analysis, and language translation tasks. Moreover, CNNs are employed in recommender systems for analyzing and processing visual content to provide personalized recommendations to users. Their versatility and effectiveness in handling complex data structures make CNNs indispensable in numerous industries, ranging from healthcare and automotive to e-commerce and entertainment, where visual data processing is paramount.

## REFERENCES


[1] Hand D. J. ,Yu. K. (2001),“Idiot's Bayes — not so stupid after all”,International Statistical Review. 69 (3): 385–399.

[2] Education Ecosystem (LEDU),“Understanding K-means Clustering in Machine Learning”, online at: https://towardsdatascience.com/understanding-k-means-clustering-in-machine-learning-6a6e67336aa1

[3] Ashley,“An Overview on Convolutional Neural Networks”,online at: https://medium.com/swlh/an-overview-on-convolutional-neural-networks-ea48e76fb186

[4] M. Jordan,J. Kleinberg,B. Scho¨lkopf,“Pattern Recognition and Machine Learning”, Springer--2006